Voor jouw apotheek

Svelte Fundamentals

Short recap day #2

Peter Kassenaar – info@kassenaar.com

eduvision
opleiding & training

Last time – 31 March

1. Working with user input and form data
   - Binding (`bind:`), validation, controlled/uncontrolled
2. Http:communicating with external API's
   - Design pattern in depth: Smart/View components
3. State management in Svelte Applications
   - Classic, using `store`
   - Modern, using global `$state()`

1. Binding to form fields

- Only on HTML elements that you use for *user input*
  - Text fields, checkboxes, radio buttons, etc
- `<input type="…" bind:xxx="…">`
  - `<input type="text" bind:value={someVariable} />`
  - `<input type="checkbox" bind:checked={someVariable}/>`
  - `<input type="radio" bind:group={someVariable} />`
  - `<input type="checkbox" bind:group={someVariable} />`
  - `<select bind:value="someVar"> + <option value="…" />`

Example binding to Form Fields

Your Profile (from workshop):

First name Peter

Last name Kassenaar

☑ Available for freelance work?

Experience level

○ Junior ○ Medior ◉ Senior

Skills

☑ HTML ☐ CSS ☐ JavaScript ☐ TypeScript ☑ Svelte

Bio

```
I am an experienced
developer,
blablabla...
```

Submit Reset

Live profile preview

Name: Peter Kassenaar

Available: true

Level: senior

Skills: HTML, Svelte

Bio:

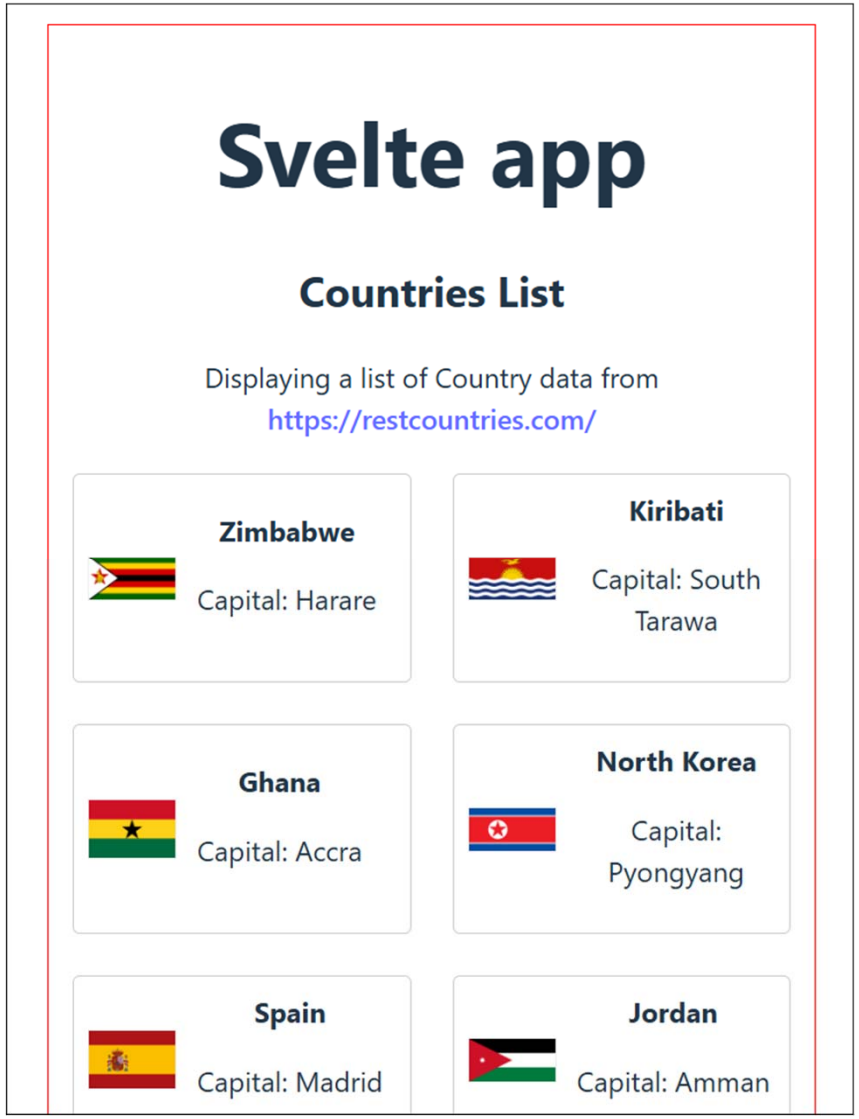
```
I am an experienced developer, blablabla...
```

…/170-bind-form-fields/../ProfileForm-workshop.svelte

2. Using HTTP

- Choices
  - Using native `fetch()`
  - Using additional library like `axios`
- Prepare state: `loading`, `done`, `error`, fill accordingly
- Possible use: lifecycle hooks (which is a *concept!*)
- Have an API available to talk to
  - GET / POST data
- Use classic Promise notation or Async/Await

Creating HTTP-calls

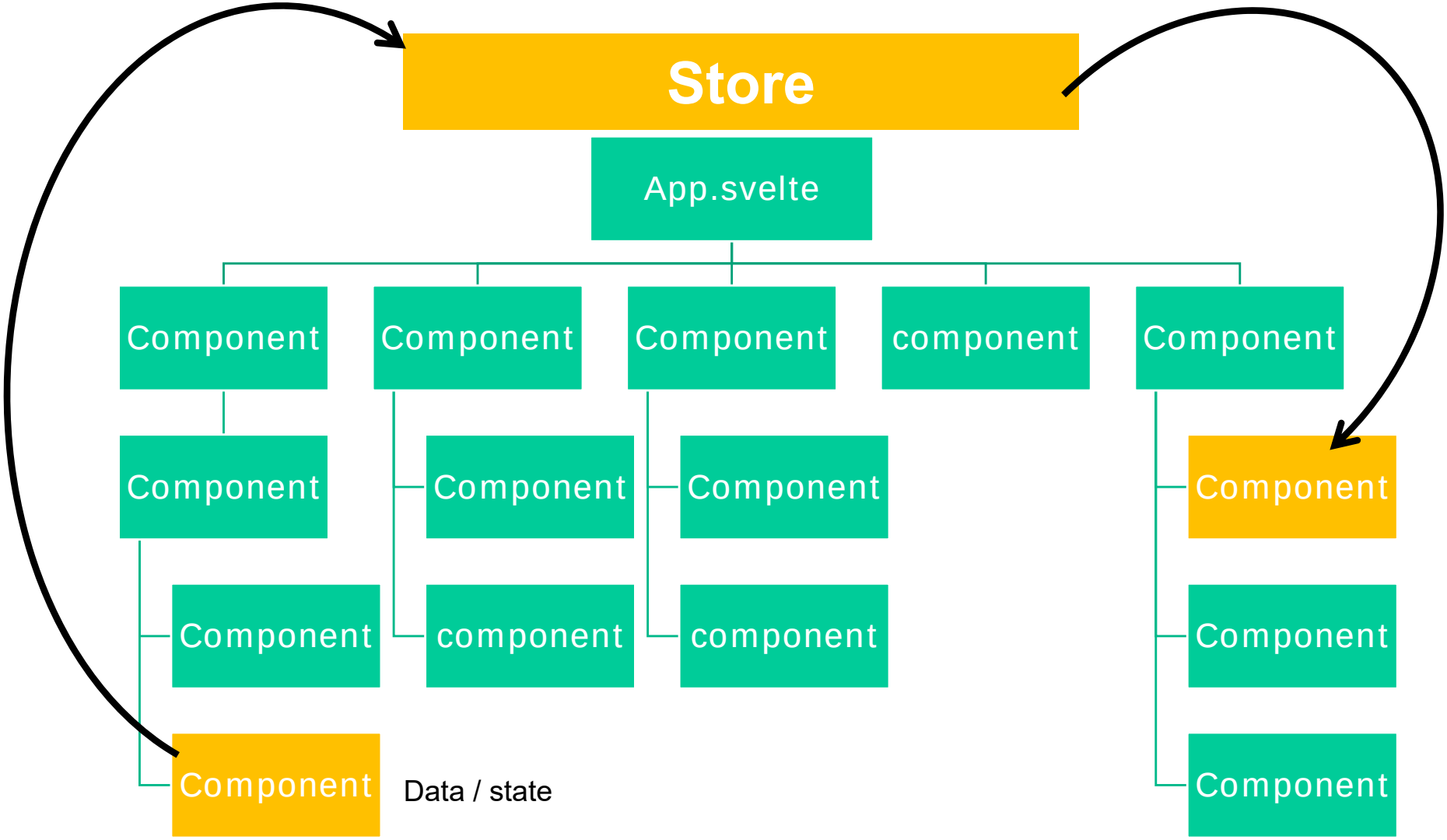


Example API: `https://restcountries.com/`

Concept: Smart Components / View Components

- Centralize logic in Smart Components
- Pass data as props to View Components
- Why? Separation of concerns
  - All components are fundamentally equal. You have to separate responsibilities yourself.
- Pass data
- Pass functions to View Components

3. State management



Multiple ways of handling state

- Classic – using `xxxStore.ts`
  - Using `writable(…)`, `.update()` and more
  - Using `$`-prefix on variables to automagically update
- Modern – using global `$state()`
  - Creating top level `xxx.svelte.ts` file, exporting the `$state()` rune
- Import the `$state()` store and its modification functions
  - Svelte: no reducers, no `<context>`, no selectors

Svelte: usually NO top-level gigantic store

- Other frameworks (React, Angular, Vue): ONE single store, holding all values
  - Frameworks: Redux, Flux, NgRX, Vuex, Pinia
- Svelte: multiple, specialized `xxxState.svelte.ts` files
- Mostly: NOT combining single stores into one super-top-level-store
  - Different mental model for Svelte

Svelte Mental Model Store Pattern

- Local component state for component-specific data
- Shared module state for app-wide or feature-wide state
- Derived state for computed values
- Stores / runes modules where shared reactivity is useful
- Context API for nested component trees
  - Not handled in this course – often less necessary
  - https://svelte.dev/docs/svelte/context

More on Svelte Context, online

SVELTE Docs Tutorial Packages Playground Blog search CTRL K

Introduction
Overview
Getting started
.svelte files
.svelte.js and .svelte.ts files

Runes
What are runes?
$state
$derived
$effect
$props
$bindable
$inspect
$host

Template syntax
Basic markup
{#if ...}
{#each ...}
{#key ...}
{#await ...}
{#snippet ...}
{@render ...}
{@html ...}
{@attach ...}
{@const ...}
{@debug ...}

SVELTE • RUNTIME

Context

Context allows components to access values owned by parent components without passing them down as props (potentially through many layers of intermediate components, known as 'prop-drilling').

By creating a `[get, set]` pair of functions with `createContext`, you can set the context in a parent component and get it in a child component:

App.svelte Parent.svelte Child.svelte context.ts Open in playground JS TS

```
import { createContext } from 'svelte';

interface User {
  name: string;
}

export const [getUserContext, setUserContext] = createContext<User>();
```

`createContext` was added in version 5.40. If you are using an earlier version of Svelte, you must use `setContext` and `getContext` instead.

This is particularly useful when `Parent.svelte` is not directly aware of `Child.svelte`, but instead renders it as part of a `children` snippet as shown above.

On this page
Context
setContext and getContext
Using context with state
Component testing
Replacing global state

https://svelte.dev/docs/svelte/context

Again, in general:

Learn Concepts

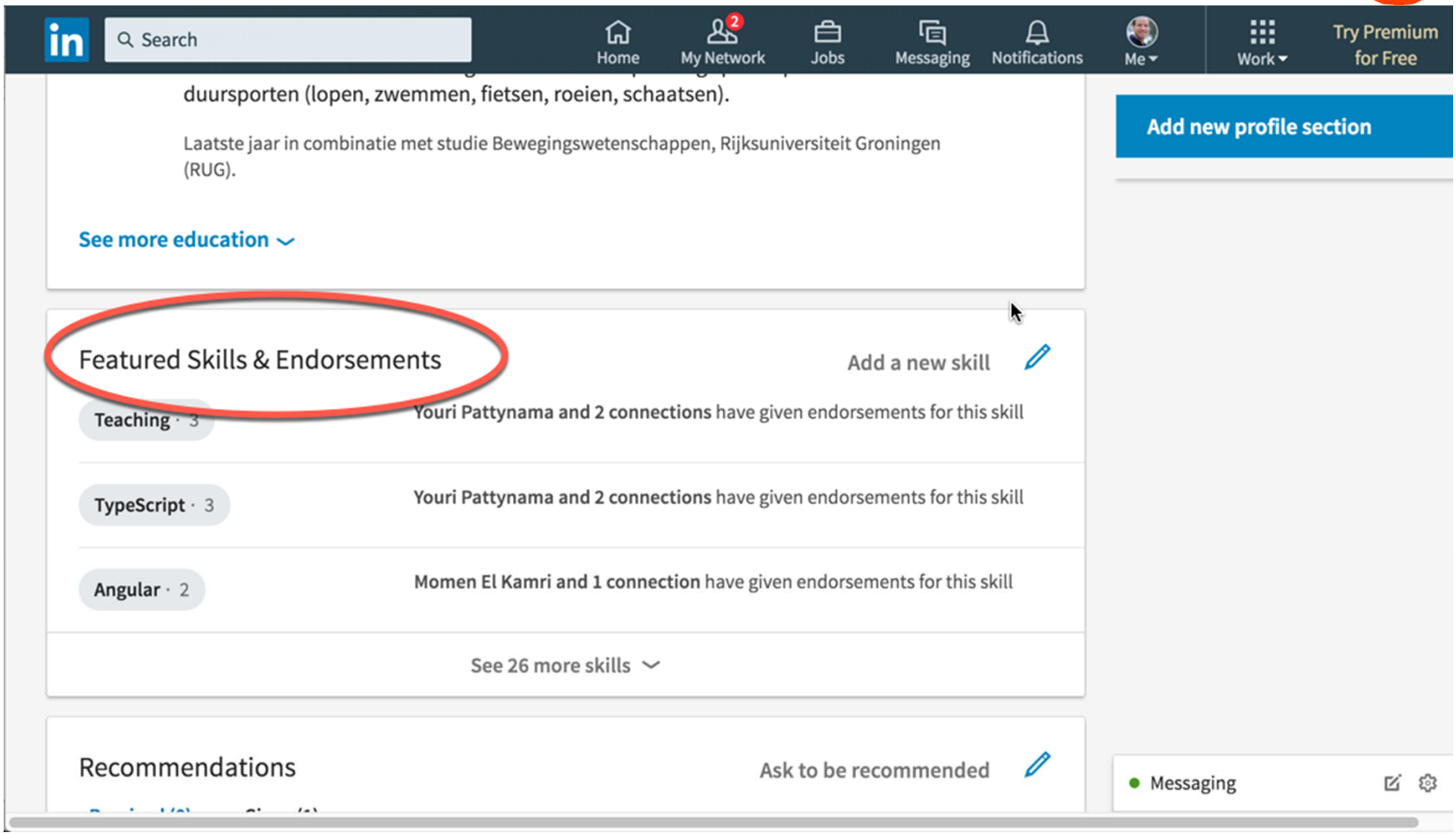
instead of Code

Questions?

Today - day 3: SvelteKit

- Goals of SvelteKit – setting up SvelteKit
- Client modules & Server modules
- File based routing
  - `+page.svelte`, `+layout.svelte`, `params`, nested layouts
- Data fetching in SvelteKit (server sided)
- Form actions and `+server.ts` API endpoints
- (Environment variables
- Deployment and strategies)

If you liked it, endorse



https://www.linkedin.com/in/peterkassenaar/

Voor jouw apotheek

Thank you

eduvision
opleiding & training